

Lab Goal : This lab was designed to teach you how to solve a weighted shortest path problem using Dijkstra's algorithm on an adjacency matrix.

Lab Description : For each graph shown here and given in the data file, you will store it in an adjacency matrix and then perform all three DFS algorithms as well as the BFS and Dijkstra's algorithms, all from the given starting vertices.

Sample Graphs and Outputs*:

Graph 1:

	A	B	C	D	E	F
A	0	0	2	8	3	0
B	0	0	0	6	0	0
C	0	3	0	4	9	0
D	0	0	0	0	0	0
E	0	0	0	0	0	0
F	1	0	0	0	0	0

DFS recursive: FACBDE

DFS iter ->: FAEDCB

DFS iter <-: FACBDE

BFS: FACDEB

Dijkstra's Algorithm starting at F:

Costs: [1, 6, 3, 7, 4, 0]

Prevs: [5, 2, 0, 2, 0, -1]

Visited: [true, true, true, true, true, true]

Path from F to A: F -> A

Path from F to B: F -> A -> C -> B

Path from F to C: F -> A -> C

Path from F to D: F -> A -> C -> D

Path from F to E: F -> A -> E

Path from F to F: F

Dijkstra's Algorithm starting at A:

Costs: [0, 5, 2, 6, 3, 2147483647]

Prevs: [-1, 2, 0, 2, 0, -1]

Visited: [true, true, true, true, true, false]

Path from A to A: A

Path from A to B: A -> C -> B

Path from A to C: A -> C

Path from A to D: A -> C -> D

Path from A to E: A -> E

Path from A to F: does not exist.

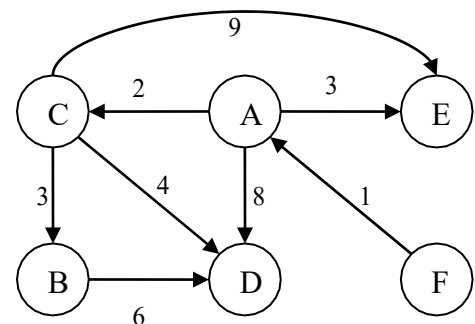
algorithm help

Before the loop,

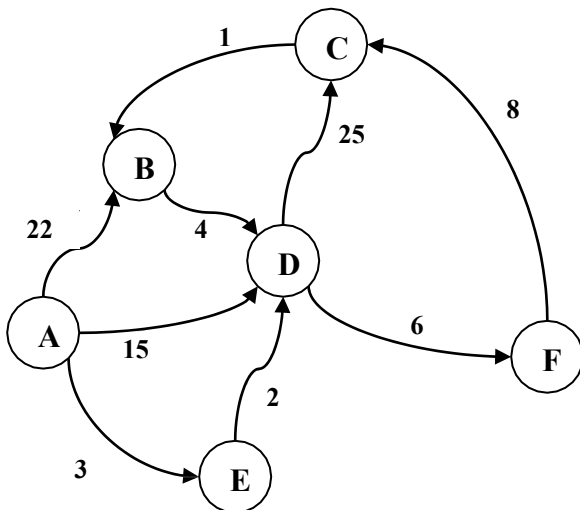
- make maps to store the costs and previous nodes and a set to keep track of the visited nodes.
- Map each vertex to a cost of Double.POSITIVE_INFINITY and previous as null.
- Set the cost of the starting node to zero.

Loop size times:

- Find the node with the smallest cost not yet visited and set it as the current node
- If there are no more reachable nodes, stop looping.
- Mark the current node as visited
- Loop thru all neighbors of the current node:
 - If the neighbor has already been visited:
 - Skip it
 - Calculate the tentative cost this neighbor
 - If the tentative cost is cheaper than the current cost stored for this neighbor:
 - Update the cheapest cost and prev



There are two text files provided called expected1.out and expected2.out. The first has the same outputs as shown here. The second shows the state of the costs, predecessors, and visited structures after every iteration of dijkstra's. Move the code the shows the state in or out of the dijkstra's loop to control how much is outputted.



Graph 2:

	A	B	C	D	E	F
A	0	22	0	15	3	0
B	0	0	0	4	0	0
C	0	1	0	0	0	0
D	0	0	25	0	0	6
E	0	0	0	2	0	0
F	0	0	8	0	0	0

DFS recursive: ABDCFE

DFS iter ->: AEDFCB

DFS iter <-: ABDCFE

BFS: DCFB

Dijkstra's Algorithm starting at A:

Costs: [0, 20, 19, 5, 3, 11]

Prevs: [-1, 2, 5, 4, 0, 3]

Visited: [true, true, true, true, true, true]

Path from A to A: A

Path from A to B: A -> E -> D -> F -> C -> B

Path from A to C: A -> E -> D -> F -> C

Path from A to D: A -> E -> D

Path from A to E: A -> E

Path from A to F: A -> E -> D -> F

Costs: [2147483647, 0, 18, 4, 2147483647, 10]

Prevs: [-1, -1, 5, 1, -1, 3]

Visited: [false, true, true, true, false, true]

Dijkstra's Algorithm starting at B:

Path from B to A: does not exist.

Path from B to B: B

Path from B to C: B -> D -> F -> C

Path from B to D: B -> D

Path from B to E: does not exist.

Path from B to F: B -> D -> F

Graph 3:

	A	B	C	D	E	F
A	0	22	0	15	3	0
B	22	0	1	4	0	0
C	0	1	0	25	0	8
D	15	4	25	0	2	6
E	3	0	0	2	0	0
F	0	0	8	6	0	0

DFS recursive: CBADEF

DFS iter ->: CFDEAB

DFS iter <-: CBADEF

BFS: FCDBAE

Dijkstra's Algorithm starting at E:

Costs: [3, 6, 7, 2, 0, 8]

Prevs: [4, 3, 1, 4, -1, 3]

Visited: [true, true, true, true, true, true]

Path from E to A: E -> A

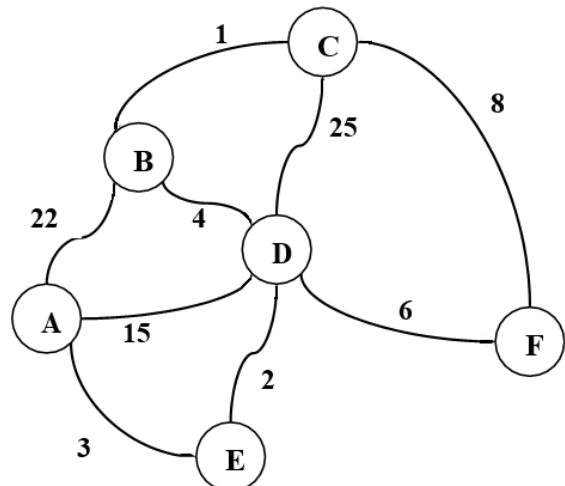
Path from E to B: E -> D -> B

Path from E to C: E -> D -> B -> C

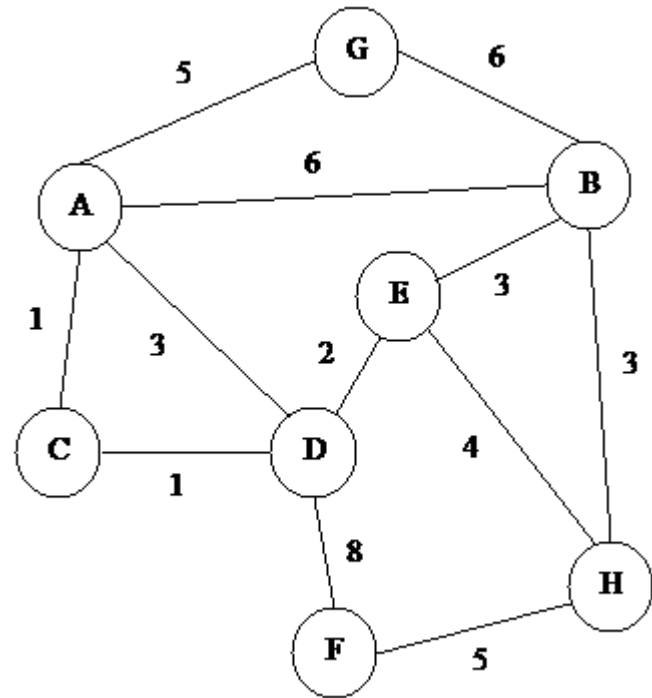
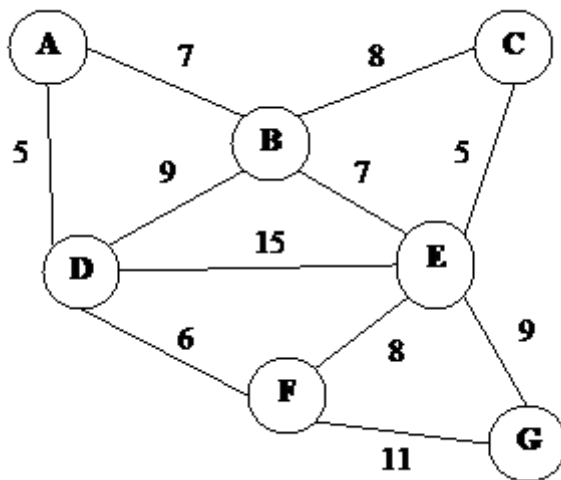
Path from E to D: E -> D

Path from E to E: E

Path from E to F: E -> D -> F



Sample Data :



Graph 4:

	A	B	C	D	E	F	G
A	0	7	0	5	0	0	0
B	7	0	8	9	7	0	0
C	0	8	0	0	5	0	0
D	5	9	0	0	15	6	0
E	0	7	5	15	0	8	9
F	0	0	0	6	8	0	11
G	0	0	0	0	9	11	0

DFS recursive: ABCEDFG

DFS iter ->: ADFGECB

DFS iter <-: ABCEDFG

BFS: ABDCEFG

Dijkstra's Algorithm starting at A:

Costs: [0, 7, 15, 5, 14, 11, 22]

Prevs: [-1, 0, 1, 0, 1, 3, 5]

Visited: [true, true, true, true, true, true, true]

Path from A to A: A

Path from A to B: A -> B

Path from A to C: A -> B -> C

Path from A to D: A -> D

Path from A to E: A -> B -> E

Path from A to F: A -> D -> F

Path from A to G: A -> D -> F -> G

Graph 5:

	A	B	C	D	E	F	G	H
A	0	6	1	3	0	0	5	0
B	6	0	0	0	3	0	6	3
C	1	0	0	1	0	0	0	0
D	3	0	1	0	2	8	0	0
E	0	3	0	2	0	0	0	4
F	0	0	0	8	0	0	0	5
G	5	6	0	0	0	0	0	0
H	0	3	0	0	4	5	0	0

DFS recursive: ABEDCFHG

DFS iter ->: AGBHFDEC

DFS iter <-: ABEDCFHG

BFS: ABCDGEHF

Dijkstra's Algorithm starting at A:

Costs: [0, 6, 1, 2, 4, 10, 5, 8]

Prevs: [-1, 0, 0, 2, 3, 3, 0, 4]

Visited: [true, true, true, true, true, true, true, true]

Path from A to A: A

Path from A to B: A -> B

Path from A to C: A -> C

Path from A to D: A -> C -> D

Path from A to E: A -> C -> D -> E

Path from A to F: A -> C -> D -> F

Path from A to G: A -> G

Path from A to H: A -> C -> D -> E -> H